

Predicting Movie Profitability from Pre-Release Metadata

A Supervised Learning Approach Using TMDB Data

Submission 1: Proposal, Code, and GitHub Repository
Machine Learning Assignment

Field	Value
Project Title	Predicting Movie Profitability from Pre-Release Metadata
Dataset Name	TMDB All Movies (Daily Updates)
Source	https://www.kaggle.com/datasets/alanvourch/tmdb-movies-daily-updates
Number of Rows	1,188,481 (raw); 13,750 (modeling subset)
Number of Columns	28 (raw); 21 (engineered features)
Target Variable	hit (binary): 1 if revenue $\geq 2 \times$ budget, else 0
Task Type	Binary Classification
Number of Research Questions	7

1. Project Title

Predicting Movie Profitability from Pre-Release Metadata: A Supervised Learning Approach Using TMDB Data

2. Dataset

Name: TMDB All Movies (Daily Updates)

Source: <https://www.kaggle.com/datasets/alanvourch/tmdb-movies-daily-updates>

Original size: 1,188,481 rows × 28 columns (≈ 696 MB CSV)

Modeling subset: 13,750 rows × 21 engineered features (after filtering for movies with budget > \$1,000 AND revenue > \$1,000)

Key columns used

- title — movie title (identifier only, not used as a feature)
- budget — production budget in USD
- revenue — worldwide gross revenue in USD
- runtime — movie length in minutes
- popularity — TMDB popularity score
- vote_average / vote_count — audience rating signals
- release_date — used to derive release_year and release_month
- genres — used to derive primary_genre and binary genre indicators
- production_companies — counted to derive num_production_companies
- original_language — used to derive is_english binary feature
- cast — counted to derive num_cast

3. Target Variable

hit (binary, 0 or 1)

Definition: hit = 1 if revenue $\geq 2 \times$ budget, else 0

Justification: The 2× rule of thumb is the industry-standard breakeven point for theatrical movies, accounting for marketing and distribution costs which typically equal the production budget. This is well-cited in the film economics literature (Vogel, Entertainment Industry Economics).

Class balance on the modeling subset: 56% flop (n=7,756), 44% hit (n=5,994). The dataset is well-balanced and does not require oversampling.

4. Type of Task

Binary Classification (Supervised Learning).

5. Problem Statement

Movie production is a capital-intensive industry where individual films routinely cost between \$1 million and \$300 million, yet only 40-45% of theatrical releases recover twice their production budget. Studios, financiers, and distributors face a high-stakes decision problem: given only pre-release information about a movie (budget, runtime, genre, language, production scale, release

timing), can the financial outcome be predicted with sufficient accuracy to inform investment decisions?

Existing approaches in the literature have used post-release signals (opening weekend numbers, early reviews) which limit their decision usefulness. This project addresses the harder pre-release prediction problem using the publicly available TMDb metadata corpus, comparing classical supervised learning algorithms and analysing where the prediction succeeds and where it fails. The findings are intended to inform both the academic understanding of movie success determinants and the practical question of whether pre-release ML prediction can support real investment decisions.

6. Research Questions

This project is guided by the following seven research questions, each addressing a distinct aspect of the supervised learning problem:

RQ1: How do classical supervised learning algorithms (Logistic Regression, Random Forest, Gradient Boosting, XGBoost, SVM) compare in predicting whether a movie will be a financial hit using only pre-release metadata?

RQ2: Which pre-release features (budget, runtime, genre, language, production scale, release timing) contribute most to predicting movie profitability?

RQ3: Does the predictability of movie success vary across budget tiers (Low, Mid, High, Blockbuster), and which tier benefits most from machine learning prediction?

RQ4: Which movie genres are most and least predictable, and how does genre interact with the determinants of hit probability?

RQ5: How stable is predictive performance when training on historical data and testing on more recent movies, and does the streaming era alter what predicts theatrical profitability?

RQ6: How does decision-threshold tuning affect the precision-recall trade-off, and what threshold maximises F1 under different cost assumptions?

RQ7: What characterises the movies the model misclassifies (false positives and false negatives), and what does this reveal about the limits of pre-release predictability?

7. Proposed Methodology

The project follows a standard supervised learning pipeline divided into seven stages:

Stage 1: Data acquisition and inspection

- Download the TMDb All Movies CSV from the public Kaggle source (1.18 million rows).
- Inspect schema, dtypes, and missingness patterns across all 28 columns.
- Document data quality observations and identify columns suitable for feature engineering.

Stage 2: Filtering and target definition

- Filter the raw dataset to retain only movies with budget > \$1,000 AND revenue > \$1,000. This removes movies for which financial outcomes are unknown or unreported, leaving 13,750 records suitable for supervised learning.
- Define the binary target hit = 1 if revenue $\geq 2 \times$ budget, else 0, following the industry-standard 2 $\times$ breakeven rule for theatrical economics.

- Verify class balance on the resulting modeling subset (44% hit, 56% flop — no resampling required).

Stage 3: Feature engineering

- Numeric transforms: $\log_budget = \log(1+budget)$, $\log_popularity = \log(1+popularity)$, to compress heavy-tailed distributions.
- Temporal features: `release_year`, `release_month` derived from `release_date`.
- Count features: `num_genres`, `num_production_companies`, `num_cast` — counted from comma-separated string fields.
- Categorical encoding: `is_english` binary indicator, plus binary indicators for the 12 most common genres (Drama, Comedy, Action, Horror, Adventure, Crime, Thriller, Animation, Romance, Science Fiction, Family, Documentary).
- Final feature matrix: 21 numeric features ready for sklearn-compatible models.

Stage 4: Train-test split

- Stratified 80/20 split using `sklearn.model_selection.train_test_split` with `random_state=42` to ensure reproducibility.
- Stratification preserves the 44% hit rate in both train and test sets.
- For RQ5 (temporal stability), the random split is replaced with chronological splits.

Stage 5: Model training

- Five supervised classifiers are trained: Logistic Regression, SVM (RBF kernel), Random Forest, Gradient Boosting, and XGBoost.
- Linear and kernel-based models receive standardised features (`StandardScaler`); tree-based models use raw features.
- Default hyperparameters are used for the baseline comparison; model-specific configurations are documented in each notebook.

Stage 6: Evaluation

- Predictions are evaluated on the held-out test set using Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
- For RQ3 and RQ4, per-segment evaluation is performed by stratifying the test set by budget tier or primary genre.
- For RQ5, forward-in-time evaluation uses three sliding train/test windows.
- For RQ6, the decision threshold is swept from 0.05 to 0.95 and precision/recall/F1 trajectories are recorded.
- For RQ7, the confusion matrix is decomposed into TP/TN/FP/FN and the average feature values per quadrant are reported.

Stage 7: Visualisation and reporting

- Each research question produces one publication-grade figure (saved as PDF and PNG) and one results table (saved as CSV).
- Figures use a consistent colour palette and matplotlib styling for visual coherence.
- All seven RQs are answered in seven self-contained Jupyter notebooks (RQ1.ipynb through RQ7.ipynb), each runnable on Kaggle without modification.

8. Machine Learning Models

Five supervised classification algorithms will be trained and compared:

Model	Library	Family	Role
Logistic Regression	sklearn.linear_model	Linear	Interpretable linear baseline
SVM (RBF kernel)	sklearn.svm	Kernel	Non-linear baseline
Random Forest	sklearn.ensemble	Bagging	Variance-reducing tree ensemble
Gradient Boosting	sklearn.ensemble	Boosting	Sequential tree ensemble
XGBoost	xgboost	Boosting	State-of-the-art gradient boosting

9. Evaluation Metrics

Five complementary classification metrics are reported for each model:

Metric	Definition	Why it matters
Accuracy	$(TP + TN) / \text{total}$	Overall correctness; easy to interpret but misleading under imbalance
Precision	$TP / (TP + FP)$	Of movies predicted to be hits, how many actually are. Critical when false positives are costly (greenlighting flops)
Recall	$TP / (TP + FN)$	Of actual hits, how many the model catches. Critical when missed hits are costly (passing on profitable scripts)
F1-Score	Harmonic mean of precision and recall	Single number that balances precision and recall; the primary headline metric
ROC-AUC	Area under the ROC curve	Threshold-independent measure of ranking quality

10. Expected Figures and Tables

Each research question produces one figure and one table. All figures are saved as PDF (publication quality) and PNG (preview), and all tables as CSV.

RQ	Figure	Table
RQ1	Grouped bar chart comparing 5 models across 5 metrics	table_rq1_model_comparison.csv
RQ2	Horizontal bar chart of top-10 feature importances	table_rq2_feature_importance.csv
RQ3	Two-panel plot: hit rate per tier + F1/AUC per tier	table_rq3_budget_tiers.csv
RQ4	Bubble plot of hit rate vs F1, sized by genre frequency	table_rq4_genre_analysis.csv
RQ5	Two-panel plot: forward-in-time bars + per-year AUC	table_rq5_temporal_stability.csv

	trend	
RQ6	Two-panel plot: PR curve with operating points + threshold sweep	table_rq6_threshold_optimization.csv
RQ7	Two-panel plot: confusion matrix + error-quadrant feature radar	table_rq7_error_analysis.csv

11. Tools and Infrastructure

- Programming language: Python 3.10+
- Numerical computing: NumPy, Pandas
- Machine learning: scikit-learn, XGBoost
- Visualisation: Matplotlib, Seaborn
- Notebook environment: Jupyter (executable on Kaggle without modification)
- Version control and submission: Git, GitHub (public repository)

12. Deliverables

- This proposal document (PDF + DOCX)
- Seven executed Jupyter notebooks (RQ1.ipynb through RQ7.ipynb) with visible outputs
- Seven figure PDFs and seven CSV tables produced by the notebooks
- Public GitHub repository containing all code, README.md, and requirements.txt
- Direct dataset link to the Kaggle source